

Reasoning about Functional Programs by Combining Interactive and Automatic Proofs

Andrés Sicard-Ramírez

Supervisors: Ana Bove and Peter Dybjer

Director of studies: Alberto Pardo

Universidad de la República

Montevideo, Uruguay

25 July 2014

Introduction

What if we have written a Haskell-like program and we want to verify it?

Introduction

What if we have written a Haskell-like program and we want to verify it?

- How to deal with the possible use of **general** recursion (non-structural recursive, nested recursive, and higher-order recursive functions, and guarded and unguarded co-recursive functions)?

Most of the proof assistants **lack a direct treatment** for general recursive functions (Bove, Krauss and Sozeau 2012).

Introduction

What if we have written a Haskell-like program and we want to verify it?

- How to deal with the possible use of **general** recursion (non-structural recursive, nested recursive, and higher-order recursive functions, and guarded and unguarded co-recursive functions)?

Most of the proof assistants **lack a direct treatment** for general recursive functions (Bove, Krauss and Sozeau 2012).

- Other features of Haskell-Like programs

Higher-order functions (in functional languages, functions can take functions as **arguments** and produce functions as **results**).

Lazy (the arguments of a function are evaluated when it is **strictly** necessary).

Inductive and co-inductive data types (**finite** and **potentially infinite** data).

Our Goal

To build a **computer-assisted** framework for reasoning about programs written in **Haskell**-like lazy functional languages.

Our Main Contributions

What programming logic should we use?

Our Main Contributions

What programming logic should we use?

We defined and formalised the First-Order Theory of Combinators

Our Main Contributions

What programming logic should we use?

We defined and formalised the First-Order Theory of Combinators

Target language	Type-free extended versions of Plotkin's PCF language
Basic data	Inductive and co-inductive data types
Specification language	First-order logic and predicates representing the property of being a finite or a potentially infinite value
The theory can deal with	General recursion, higher-order functions, (co-)inductive definitions of data types and proofs by (co-)induction
Consistency	Based on a translation into Dybjer's (1985) Logical Theory of Constructions

Our Main Contributions

What proof assistant should we use?

We formalise our programming logics and our examples of verification of functional programs in the **Agda** proof assistant:

Our Main Contributions

What proof assistant should we use?

We formalise our programming logics and our examples of verification of functional programs in the **Agda** proof assistant:

- we use **Agda** as a **logical framework** (meta-logical system for formalising other logics) and
- we use **Agda's proof engine**: (i) support for inductively defined types, including inductive families, and function definitions using pattern matching on such types, (ii) normalisation during type-checking, (iii) commands for refining proof terms, (iv) coverage checker and (v) termination checker.

Our Main Contributions

Can (part of) the job be automatic?

Yes! We can combine **Agda** interactive proofs and ATPs (automatic theorem provers for first-order logic) proofs:

Our Main Contributions

Can (part of) the job be automatic?

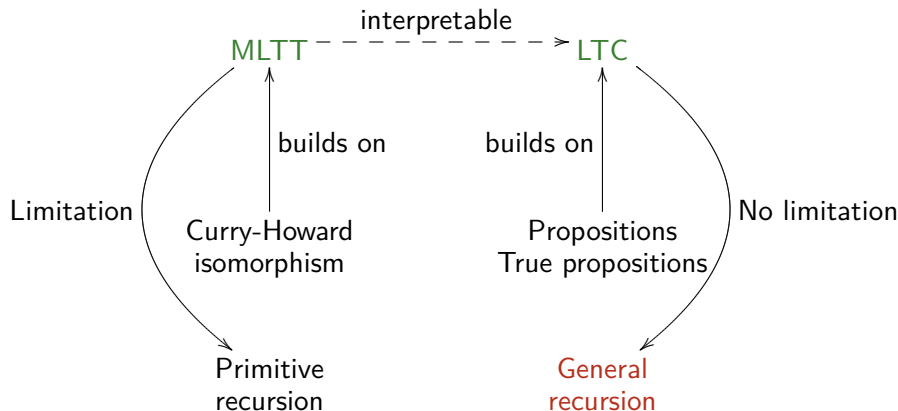
Yes! We can combine **Agda** interactive proofs and ATPs (automatic theorem provers for first-order logic) proofs:

- we provide a translation of our **Agda** representation of first-order formulae into **TPTP** (Sutcliffe 2009)—a language understood by many off-the-shelf ATPs—so we can use them when proving the properties of our programs,
- we extended **Agda** with an ATP-pragma, which instructs **Agda** to interact with the ATPs, and
- we wrote the **Apia** program, a **Haskell** program which uses **Agda** as a **Haskell** library, performs the above translation and calls the ATPs.

Combining Three Strands of Research

1. Foundational frameworks and logics for lazy functional programs

Why use LTC as a programming logics for lazy functional programs (Dybjer 1985, 1990; Dybjer and Sander 1989)



Combining Three Strands of Research

2. Proving correctness of functional programs using first-order automatic theorem provers

“The CoVer Translator” (Claessen and Hamon 2003)

Using ATPs for proving properties of functional programs by translating them into first-order logic.

Combining Three Strands of Research

2. Proving correctness of functional programs using first-order automatic theorem provers

“The CoVer Translator” (Claessen and Hamon 2003)

Using ATPs for proving properties of functional programs by translating them into first-order logic.

3. Connecting first-order automatic theorem provers to type theory systems

The implementation of the **Apia** program took some ideas from the connection of **AgdaLight** (an experimental version of **Agda**) to the **Gandalf** ATP (Abel, Coquand and Norell 2005).

First-Order Logic

Terms $\ni t ::= x$ variable
 $| c$ constant
 $| f(t, \dots, t)$ function

Formulae $\ni A ::= \top \mid \perp$ truth, falsehood
 $| A \supset A \mid A \wedge A \mid A \vee A$ binary logical connectives
 $| \forall x.A \mid \exists x.A$ quantifiers
 $| t = t$ equality
 $| P(t, \dots, t)$ predicate

Abbreviations

$\neg A \stackrel{\text{def}}{=} A \supset \perp$ negation
 $t \neq t' \stackrel{\text{def}}{=} \neg(t = t')$ inequality

Formalising First-Order Logic

Using Agda as an logical framework

- Edinburgh Logical Framework (LF) approach

We **postulate** each logical constant as a type former, and each axiom and inference rule as a constants of the corresponding type.

Formalising First-Order Logic

Using Agda as an logical framework

- Edinburgh Logical Framework (LF) approach

We **postulate** each logical constant as a type former, and each axiom and inference rule as a constants of the corresponding type.

- Basic inductive approach

The introduction rules of the logical constants are represented by **inductive types**, and their elimination rules are defined by **pattern matching**.

Formalising First-Order Logic

Using Agda as an logical framework

- Edinburgh Logical Framework (LF) approach

We **postulate** each logical constant as a type former, and each axiom and inference rule as a constants of the corresponding type.

- Basic inductive approach

The introduction rules of the logical constants are represented by **inductive types**, and their elimination rules are defined by **pattern matching**.

- Inductive approach

To make **full** use of **Agda**'s support for proof by pattern matching, we shall allow proofs by pattern matching in general (not only for the elimination rules), as long as they are accepted by **Agda**'s coverage and termination checker.

Formalising First-Order Logic

Example (existential quantifier)

$$\frac{A(t)}{\exists x.A(x)} (\exists\text{I}) \qquad \frac{\begin{array}{c} [A] \\ \vdots \\ \exists x.A(x) \end{array} \quad B}{B} (\exists\text{E})$$

(side condition for the rule $\exists\text{E}$: x is not free in B or in any of the assumptions of the proof of B other than $A(x)$)

Formalising First-Order Logic

Example (existential quantifier (cont.))

LF- and inductive approaches

Domain of quantification

postulate D : **Set**

Formalising First-Order Logic

Example (existential quantifier (cont.))

LF- and inductive approaches

Domain of quantification

postulate $D : \mathbf{Set}$

LF-approach

postulate

$\exists$: $(A : D \rightarrow \mathbf{Set}) \rightarrow \mathbf{Set}$
 $_ , _$: $\{A : D \rightarrow \mathbf{Set}\} (t : D) \rightarrow$
 $A \ t \rightarrow \exists A$
 $\exists$ -elim : $\{A : D \rightarrow \mathbf{Set}\} \{B : \mathbf{Set}\} \rightarrow$
 $\exists A \rightarrow$
 $(\forall \{x\} \rightarrow A \ x \rightarrow B) \rightarrow B$

Formalising First-Order Logic

Example (existential quantifier (cont.))

LF- and inductive approaches

Domain of quantification

```
postulate D : Set
```

LF-approach

```
postulate
```

```
  ∃      : (A : D → Set) → Set
  _,_    : {A : D → Set}{t : D} →
           A t → ∃ A
  ∃-elim : {A : D → Set}{B : Set} →
           ∃ A →
           (∀ {x} → A x → B) → B
```

Inductive approaches

```
data ∃ (A : D → Set) : Set where
```

```
  _,_ : (t : D) → A t → ∃ A
  ∃-elim : {A : D → Set}{B : Set} →
           ∃ A → (∀ {x} → A x → B) → B
  ∃-elim (_ , Ax) h = h Ax
```

Formalising First-Order Logic

Notation: It is possible to replace $\exists (\lambda x \rightarrow e)$ by $\exists [x] e$.

Formalising First-Order Logic

Notation: It is possible to replace $\exists (\lambda x \rightarrow e)$ by $\exists [x] e$.

Example

Let $A(x, y)$ be a propositional function. The proof of

$$\exists x. \forall y. A(x, y) \supset \forall y. \exists x. A(x, y),$$

is represented as follows.

Formalising First-Order Logic

Notation: It is possible to replace $\exists (\lambda x \rightarrow e)$ by $\exists [x] e$.

Example

Let $A(x, y)$ be a propositional function. The proof of

$$\exists x. \forall y. A(x, y) \supset \forall y. \exists x. A(x, y),$$

is represented as follows.

The theorem:

$$\exists \forall : \{A : D \rightarrow D \rightarrow \mathbf{Set}\} \rightarrow \exists [x] (\forall y \rightarrow A \ x \ y) \rightarrow \forall y \rightarrow \exists [x] A \ x \ y$$

Formalising First-Order Logic

Notation: It is possible to replace $\exists (\lambda x \rightarrow e)$ by $\exists [x] e$.

Example

Let $A(x, y)$ be a propositional function. The proof of

$$\exists x. \forall y. A(x, y) \supset \forall y. \exists x. A(x, y),$$

is represented as follows.

The theorem:

$$\exists \forall : \{A : D \rightarrow D \rightarrow \mathbf{Set}\} \rightarrow \exists [x] (\forall y \rightarrow A \ x \ y) \rightarrow \forall y \rightarrow \exists [x] A \ x \ y$$

LF- and basic inductive approach proof:

$$\exists \forall \ h \ y = \exists\text{-elim } h \ (\lambda \{x\} \ ah \rightarrow x \ , \ ah \ y)$$

Formalising First-Order Logic

Notation: It is possible to replace $\exists (\lambda x \rightarrow e)$ by $\exists [x] e$.

Example

Let $A(x, y)$ be a propositional function. The proof of

$$\exists x. \forall y. A(x, y) \supset \forall y. \exists x. A(x, y),$$

is represented as follows.

The theorem:

$$\exists \forall : \{A : D \rightarrow D \rightarrow \mathbf{Set}\} \rightarrow \exists [x] (\forall y \rightarrow A \ x \ y) \rightarrow \forall y \rightarrow \exists [x] A \ x \ y$$

LF- and basic inductive approach proof:

$$\exists \forall \ h \ y = \exists\text{-elim } h \ (\lambda \{x\} \ ah \rightarrow x \ , \ ah \ y)$$

Inductive approach proof:

$$\exists \forall \ (x \ , \ Ax) \ y = x \ , \ Ax \ y$$

Our Representation of First-Order Logic

Falsehood

```
data  $\perp$  : Set where  
  
 $\perp$ -elim : {A : Set}  $\rightarrow \perp \rightarrow A$   
 $\perp$ -elim ()
```

Truth

```
data  $\top$  : Set where tt :  $\top$ 
```

Disjunction

```
data  $\_ \vee \_$  (A B : Set) : Set where  
  inj1 : A  $\rightarrow$  A  $\vee$  B  
  inj2 : B  $\rightarrow$  A  $\vee$  B  
  
case :  $\forall$  {A B}  $\rightarrow$  {C : Set}  $\rightarrow$  (A  $\rightarrow$  C)  $\rightarrow$  (B  $\rightarrow$  C)  $\rightarrow$  A  $\vee$  B  $\rightarrow$  C  
case f g (inj1 a) = f a  
case f g (inj2 b) = g b
```

Conjunction

```
data  $\_ \wedge \_$  (A B : Set) : Set where  
   $\_ , \_$  : A  $\rightarrow$  B  $\rightarrow$  A  $\wedge$  B  
  
 $\wedge$ -proj1 :  $\forall$  {A B}  $\rightarrow$  A  $\wedge$  B  $\rightarrow$  A  
 $\wedge$ -proj1 (a ,  $\_$ ) = a  
  
 $\wedge$ -proj2 :  $\forall$  {A B}  $\rightarrow$  A  $\wedge$  B  $\rightarrow$  B  
 $\wedge$ -proj2 ( $\_$  , b) = b
```

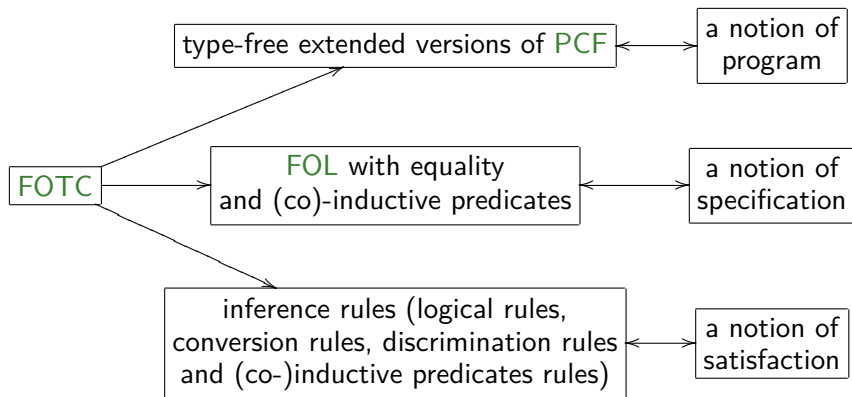
Our Representation of First-Order Logic

Conditional	$A \rightarrow B$ (non-dependent function type)
Negation	$\neg_ : \mathbf{Set} \rightarrow \mathbf{Set}$ $\neg A = A \rightarrow \perp$
Principle of the excluded middle	postulate pem : $\forall \{A\} \rightarrow A \vee \neg A$
Domain of discourse	postulate D : Set
Universal quantifier	$(x : D) \rightarrow A$ (dependent function type)
Existential quantifier	data $\exists (A : D \rightarrow \mathbf{Set}) : \mathbf{Set}$ where $_,_ : (t : D) \rightarrow A\ t \rightarrow \exists A$ $\exists$ -elim : $\{A : D \rightarrow \mathbf{Set}\}\{B : \mathbf{Set}\} \rightarrow$ $\exists A \rightarrow (\forall \{x\} \rightarrow A\ x \rightarrow B) \rightarrow B$ $\exists$ -elim ($_, Ax$) $h = h\ Ax$
Equality	data $_ \equiv_ (x : D) : D \rightarrow \mathbf{Set}$ where refl : $x \equiv x$ subst : $(A : D \rightarrow \mathbf{Set}) \rightarrow \forall \{x\ y\} \rightarrow x \equiv y \rightarrow$ $A\ x \rightarrow A\ y$ subst A refl Ax = Ax

First-Order Theory of Combinators (FOTC)

The FOTC programming logic

We extended and formalised Dybjer's (1985) Logical Theory of Constructions for extended versions of PCF.



The Programming Language of FOTC

FOTC-terms

$t ::= x$	variable
$t \cdot t$	application
true false if	partial Boolean constants
0 succ pred iszero	partial natural number constants
f	function constant

where f is a new combinator defined by a (recursive) equation

$$f \cdot x_1 \cdot \dots \cdot x_n = e[f, x_1, \dots, x_n].$$

The Specification Language of FOTC

FOTC-formulae

$A ::= \top \mid \perp$	truth, falsehood
$\mid A \supset A \mid A \wedge A \mid A \vee A$	binary logical connectives
$\mid \forall x.A \mid \exists x.A$	quantifiers
$\mid t = t$	equality
$\mid P(t, \dots, t)$	predicate
$\mid \mathit{Bool}(t)$	total Booleans predicate
$\mid \mathcal{N}(t)$	total natural numbers predicate

The Specification Language of FOTC

Inductive predicates

Bool and *N*: unary inductive predicate symbols

Bool(*t*): *t* is a total and finite Boolean value (**true** or **false**)

N(*t*): *t* is a total and finite natural number

The Specification Language of FOTC

Inductive predicates

Bool and *N*: unary inductive predicate symbols

Bool(*t*): *t* is a total and finite Boolean value (**true** or **false**)

N(*t*): *t* is a total and finite natural number

Example

We express that a function *f* terminates and it maps a total and finite natural number to a total and finite natural number by the formula

$$\forall t. \mathcal{N}(t) \supset \mathcal{N}(f \cdot t).$$

Conversion and Discrimination Rules of FOTC

Conversion rules

$$\begin{aligned} &\forall t\ t'. \text{if} \cdot \text{true} \cdot t \cdot t' = t, \\ &\forall t\ t'. \text{if} \cdot \text{false} \cdot t \cdot t' = t', \\ &\quad \text{pred} \cdot 0 = 0, \\ &\forall t. \text{pred} \cdot (\text{succ} \cdot t) = t, \\ &\quad \text{iszero} \cdot 0 = \text{true}, \\ &\forall t. \text{iszero} \cdot (\text{succ} \cdot t) = \text{false}. \end{aligned}$$

Discrimination rules for constructors

$$\begin{aligned} &\text{true} \neq \text{false}, \\ &\text{and } \forall t. 0 \neq \text{succ} \cdot t. \end{aligned}$$

The Inductive Predicate Rules of FOTC

Introduction and elimination rules for the inductive predicates $\mathcal{B}ool$ and $\mathcal{N}$

$$\begin{array}{c} \frac{}{\mathcal{B}ool(\mathbf{true})} \qquad \frac{}{\mathcal{B}ool(\mathbf{false})} \qquad \frac{\mathcal{B}ool(t) \quad A(\mathbf{true}) \quad A(\mathbf{false})}{A(t)} \\[2em] \frac{}{\mathcal{N}(0)} \qquad \frac{\mathcal{N}(t)}{\mathcal{N}(\mathbf{succ} \cdot t)} \qquad \frac{\mathcal{N}(t) \quad A(0) \quad A(\mathbf{succ} \cdot t') \quad \begin{array}{c} [A(t')] \\ \vdots \end{array}}{A(t)} \end{array}$$

Inductive Representation of FOTC

FOTC-terms

The domain universe and the term constructors are formalised by the following postulates:

postulate

```
D                               : Set
_'._                             : D → D → D
true false if                   : D
zero succ pred iszero          : D
```

Inductive Representation of FOTC

Conversion rules

The conversion rules are formalised by the following postulates:

postulate

if-true : $\forall t \{t'\} \rightarrow \text{if} \cdot \text{true} \cdot t \cdot t' \equiv t$

if-false : $\forall \{t\} t' \rightarrow \text{if} \cdot \text{false} \cdot t \cdot t' \equiv t'$

pred-0 : $\text{pred} \cdot \text{zero} \equiv \text{zero}$

pred-S : $\forall n \rightarrow \text{pred} \cdot (\text{succ} \cdot n) \equiv n$

iszero-0 : $\text{iszero} \cdot \text{zero} \equiv \text{true}$

iszero-S : $\forall n \rightarrow \text{iszero} \cdot (\text{succ} \cdot n) \equiv \text{false}$

Inductive Representation of FOTC

Conversion rules

The conversion rules are formalised by the following postulates:

postulate

if-true : $\forall t \{t'\} \rightarrow \text{if} \cdot \text{true} \cdot t \cdot t' \equiv t$
if-false : $\forall \{t\} t' \rightarrow \text{if} \cdot \text{false} \cdot t \cdot t' \equiv t'$
pred-0 : $\text{pred} \cdot \text{zero} \equiv \text{zero}$
pred-S : $\forall n \rightarrow \text{pred} \cdot (\text{succ} \cdot n) \equiv n$
iszero-0 : $\text{iszero} \cdot \text{zero} \equiv \text{true}$
iszero-S : $\forall n \rightarrow \text{iszero} \cdot (\text{succ} \cdot n) \equiv \text{false}$

Discrimination rules

The discrimination rules are formalised by the following postulates:

postulate

$t \neq f$: $\text{true} \neq \text{false}$
 $0 \neq S$: $\forall \{n\} \rightarrow \text{zero} \neq \text{succ} \cdot n$

Inductive Representation of FOTC

Classical predicate logic with equality

We use the inductive representation of FOL for representing the classical predicate logic of FOTC.

Inductive Representation of FOTC

Inference rules for the total and finite natural numbers predicate

The inductive predicate $\mathcal{N}$ is represented as an inductive family:

data $N : D \rightarrow \mathbf{Set}$ **where**

$nzero : N \text{ zero}$

$nsucc : \forall \{n\} \rightarrow N \ n \rightarrow N \ (succ \cdot n)$

Inductive Representation of FOTC

Inference rules for the total and finite natural numbers predicate

The inductive predicate $\mathcal{N}$ is represented as an inductive family:

```
data N : D → Set where  
  nzero : N zero  
  nsucc  : ∀ {n} → N n → N (succ · n)
```

We define the elimination rule for $\mathcal{N}$ by pattern matching:

```
N-ind : (A : D → Set) →  
  A zero →  
  (∀ {n} → A n → A (succ · n)) →  
  ∀ {n} → N n → A n  
N-ind A A0 h nzero      = A0  
N-ind A A0 h (nsucc Nn) = h (N-ind A A0 h Nn)
```

Inductive Representation of FOTC

Convention

Instead of using the constants `if`, `succ`, `pred` and `iszero` of type `D`, we define more readable and writable function symbols of the appropriate types.

```
if_then_else_ : D → D → D → D  
if b then t else t' = if · b · t · t'
```

```
succ1 : D → D  
succ1 n = succ · n
```

```
pred1 : D → D  
pred1 n = pred · n
```

```
iszero1 : D → D  
iszero1 n = iszero · n
```

Proving Properties by Structural Recursion

Example (addition is terminating)

The addition of total and finite natural numbers terminates.

The recursive equation:

postulate

$_+ _ : D \rightarrow D \rightarrow D$

$+ - 0x : \forall n \rightarrow \text{zero} + n \equiv n$

$+ - Sx : \forall m n \rightarrow \text{succ}_1 m + n \equiv \text{succ}_1 (m + n)$

Proving Properties by Structural Recursion

Example (addition is terminating)

The property:

$$+-N : \forall \{m\ n\} \rightarrow N\ m \rightarrow N\ n \rightarrow N\ (m + n)$$

The proof is by pattern matching on the first explicit argument:

Base case:

$$+-N \{n = n\} \text{ nzero } Nn = \text{subst } N\ (\text{sym } (+\text{-leftIdentity } n))\ Nn$$

Inductive case:

$$\begin{aligned} +-N \{n = n\} (\text{nsucc } \{m\} Nm) Nn = \\ \text{subst } N\ (\text{sym } (+\text{-Sx } m\ n))\ (\text{nsucc } (+-N\ Nm\ Nn)) \end{aligned}$$

Representation of Higher-Order Functions in FOTC

Using FOTC binary application symbol

$$_ \cdot _ : D \rightarrow D \rightarrow D$$

we can represent higher-order functions.

Example

The higher-order function that applies a unary function twice is formalised by the axioms

$$\text{twice} : D \rightarrow D \rightarrow D$$
$$\text{twice } f \ x = f \cdot (f \cdot x)$$

Adding (Co-)Inductive Predicates to FOTC

FOTC is not one first-order theory, but a family of first-order theories

- We work with one FOTC for each verification problem
- The function symbols are determined by the program we want to verify
- The predicate symbols are determined by the (co-)inductively defined predicates we need in our proofs, which can be added to FOTC under certain conditions.

Adding Inductive Predicates to FOTC

The inductively defined predicates might not only be used for representing totality properties.

Example (even predicate)

$$\frac{}{\mathcal{E}ven(0)} , \quad \frac{\mathcal{E}ven(t)}{\mathcal{E}ven(\text{succ} \cdot (\text{succ} \cdot t))} ,$$
$$\frac{\begin{array}{c} [A(t')] \\ \vdots \end{array} \quad \mathcal{E}ven(t) \quad A(0) \quad A(\text{succ} \cdot (\text{succ} \cdot t'))}{A(t)} .$$

Adding Inductive Predicates to FOTC

Example (FOTC elements for working with lists)

To use lists we add the following elements:

FOTC-terms

$$\{[], \text{cons}, \text{null}, \text{head}, \text{tail}\}.$$

Conversion rules

$$\begin{aligned} \text{null} \cdot [] &= \text{true}, \\ \forall t \, ts. \text{null} \cdot (\text{cons} \cdot t \cdot ts) &= \text{false}, \\ \forall t \, ts. \text{head} \cdot (\text{cons} \cdot t \cdot ts) &= t, \\ \forall t \, ts. \text{tail} \cdot (\text{cons} \cdot t \cdot ts) &= ts. \end{aligned}$$

Discrimination rule

$$\forall t \, ts. [] \neq \text{cons} \cdot t \cdot ts.$$

Adding Inductive Predicates to FOTC

Example (representation of the *List* predicate)

The unary inductive predicate *List*(*ts*) representing that *ts* is a total and finite list of elements.

```
data List : D → Set where  
  lnil   : List []  
  lcons  : ∀ x {xs} → List xs → List (x :: xs)
```

where

```
_::_ : D → D → D  
x :: xs = cons · x · xs
```

Remark: It is not necessary to implement the elimination rule of *List* because we shall use Agda's pattern matching instead.

Adding Co-Inductive Predicates

Example (co-natural numbers)

We implement a co-inductive predicate $\mathcal{Conat}(t)$ representing that t is potentially infinite natural number.

The unary predicate:

postulate Conat : D → Set

The unfolding rule:

postulate

Conat-out : $\forall \{n\} \rightarrow \text{Conat } n \rightarrow$
 $n \equiv \text{zero} \vee (\exists [n'] \text{ } n \equiv \text{succ}_1 \text{ } n' \wedge \text{Conat } n')$

The co-induction rule:

postulate

Conat-coind : $(A : D \rightarrow \text{Set}) \rightarrow$
 $(\forall \{n\} \rightarrow A \text{ } n \rightarrow$
 $n \equiv \text{zero} \vee (\exists [n'] \text{ } n \equiv \text{succ}_1 \text{ } n' \wedge A \text{ } n')) \rightarrow$
 $\forall \{n\} \rightarrow A \text{ } n \rightarrow \text{Conat } n$

Adding Co-Inductive Predicates

Example (streams)

We implement a co-inductive predicate representing potentially infinite list.

The unary predicate:

postulate Stream : D → **Set**

The unfolding rule:

postulate

Stream-out : $\forall \{xs\} \rightarrow \text{Stream } xs \rightarrow$
 $\exists [x'] \exists [xs'] \quad xs \equiv x' :: xs' \wedge \text{Stream } xs'$

The co-induction rule:

postulate

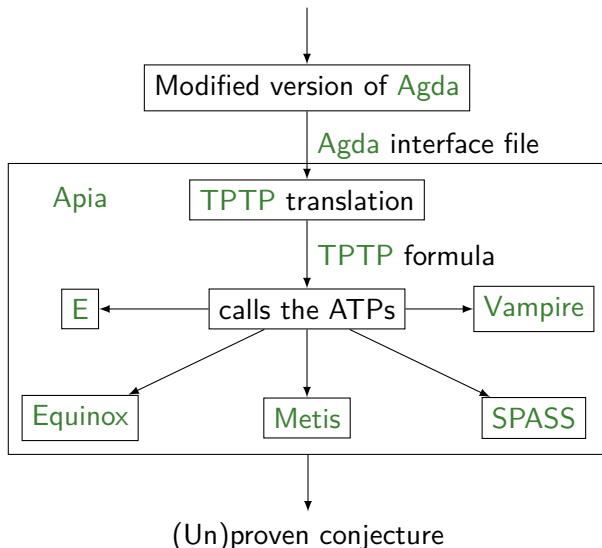
Stream-coind : (A : D → **Set**) →
($\forall \{xs\} \rightarrow A \ xs \rightarrow$
 $\exists [x'] \exists [xs'] \quad xs \equiv x' :: xs' \wedge A \ xs'$) →
 $\forall \{xs\} \rightarrow A \ xs \rightarrow \text{Stream } xs$

Combining Interactive and Automatic Proofs

- The verification of lazy functional programs requires the use of simple **equational reasoning** or simple **first-order reasoning** (low level reasoning)
- Much of this low-level reasoning can be done **automatically** with the help of, for example, automatic theorem provers for **FOL**
- By staying **strictly** within **FOL**, we shall be able to employ powerful ATPs for reasoning about functional programs

Extended Version of Agda, Apia and ATPs

Agda file + ATP-pragmas + [logical schemata options]



The TPTP Language

In **TPTP** syntax, each problem contains one or more annotated formulae of the form

$$\text{fof}(\text{name}, \text{role}, \text{formula})$$

where `name` identifies the formula within the problem, `formula` is a **FOL**-formula and `role` can be:

- conjectures: formulae to be proved
- axioms: formulae without proofs
- hypotheses: formulae assumed to be true
- definitions: formulae used to introduce symbols

Using the ATP-Pragma

ATP axioms

We tell the ATPs that the formulae A, B and C are **axioms** by

```
{-# ATP axiom A B C #-}
```

Using the ATP-Pragma

ATP axioms

We tell the ATPs that the formulae A, B and C are **axioms** by

```
{-# ATP axiom A B C #-}
```

ATP conjectures

To automatically prove a formula A, we shall **postulate** it and add the ATP-pragma

```
{-# ATP prove A #-}
```

that instructs the ATPs to prove the conjecture A.

Using the Apia Program

Example (commutativity of disjunction)

1. Postulating the property

postulate

A B : **Set**

v-comm : $A \vee B \rightarrow B \vee A$

Using the Apia Program

Example (commutativity of disjunction)

1. Postulating the property

postulate

A B : **Set**

v-comm : $A \vee B \rightarrow B \vee A$

2. Adding the ATP-pragma

{-# ATP prove v-comm #-}

Using the Apia Program

Example (commutativity of disjunction)

1. Postulating the property

postulate

A B : **Set**

v-comm : $A \vee B \rightarrow B \vee A$

2. Adding the ATP-pragma

{-# ATP prove v-comm #-}

3. Type-checking the program using Agda

\$ agda CommDisjunction.agda

Using the Apia Program

Example (commutativity of disjunction)

1. Postulating the property

postulate

$A\ B$: **Set**

$v\text{-comm}$: $A\ v\ B \rightarrow B\ v\ A$

2. Adding the ATP-pragma

{-# ATP prove $v\text{-comm}$ #-}

3. Type-checking the program using Agda

\$ agda CommDisjunction.agda

4. Proving the conjecture using Apia

\$ apia CommDisjunction.agda

Proving the conjecture in /tmp/CommDisjunction/10-8744-comm.tptp
Vampire 0.6 (revision 903) proved the conjecture

Using the Apia Program

Some command-line options

```
$ apia --help
```

```
Usage: apia [OPTIONS] FILE
```

```
  --atp=NAME      Set the ATP (e, equinox, ileancop, metis,  
                  spass, vampire)  
                  (default: e, equinox, and vampire).  
  --dump-agdai    Dump the Agda interface file to stdout.  
  --only-files    Do not call the ATPs, only to create the  
                  TPTP files.  
  --time=NUM      Set timeout for the ATPs in seconds  
                  (default: 240).
```

Trust of our Approach

- We use the ATPs as oracles via the **Apia** program
- The user must:
 - i) to add to the **Agda** program the required ATP-pragmas,
 - ii) to run the **Apia** program on the corresponding **Agda** file and
 - iii) to verify that some ATP could prove the formula.
- Implementation of the ATPs
- Implementation of **Apia**

Automatic Proofs in (Classical) First-Order Logic

Example (the principle of the exclude middle)

```
postulate pem :  $\forall \{A\} \rightarrow A \vee \neg A$   
{-# ATP prove pem #-}
```

Example (principle of the indirect proof)

```
postulate  $\neg$ -elim :  $\forall \{A\} \rightarrow (\neg A \rightarrow \perp) \rightarrow A$   
{-# ATP prove  $\neg$ -elim #-}
```

Combined Proofs in the First-Order Theory of Combinators

General methodology

We inform the ATPs that:

1. The conversion and discrimination rules associated with the **FOTC**-terms are ATP axioms

Combined Proofs in the First-Order Theory of Combinators

General methodology

We inform the ATPs that:

1. The conversion and discrimination rules associated with the **FOTC**-terms are ATP axioms
2. Each new added recursive equation is an ATP axiom. For example,

postulate

$_+ _ : D \rightarrow D \rightarrow D$

$+ - 0x : \forall n \rightarrow \text{zero} + n \equiv n$

$+ - Sx : \forall m\ n \rightarrow \text{succ}_1\ m + n \equiv \text{succ}_1\ (m + n)$

$\{-\# \text{ ATP axiom } + - 0x\ + - Sx\ \#\}$

Combined Proofs in the First-Order Theory of Combinators

General methodology

We inform the ATPs that:

3. The inductive data type constructors of the inductive predicates are ATP axioms. For example,

```
data N : D → Set where  
  nzero : N zero  
  nsucc  : ∀ {n} → N n → N (succ1 n)  
{-# ATP axiom nzero nsucc #-}
```

Combined Proofs in the First-Order Theory of Combinators

General methodology

We inform the ATPs that:

3. The inductive data type constructors of the inductive predicates are ATP axioms. For example,

```
data N : D → Set where  
  nzero : N zero  
  nsucc  : ∀ {n} → N n → N (succ1 n)  
{-# ATP axiom nzero nsucc #-}
```

4. The unfolding rule of the co-inductive predicate is an ATP axiom. For example,

```
Conat-out : ∀ {n} → Conat n →  
             n ≡ zero ∨ (∃[ n' ] n ≡ succ1 n' ∧ Conat n')  
{-# ATP axiom Conat-out #-}
```

Combined Inductive Proofs in FOTC

Example (addition is terminating)

$\text{+-N} : \forall \{m\ n\} \rightarrow N\ m \rightarrow N\ n \rightarrow N\ (m + n)$

The proof is by pattern matching on the first explicit argument.

Base case:

$\text{+-N} \{n = n\} \text{ nzero } Nn = \text{prf}$
where postulate $\text{prf} : N\ (\text{zero} + n)$
 $\{-\# \text{ ATP prove prf \#-}\}$

Inductive case:

$\text{+-N} \{n = n\} (\text{nsucc } \{m\} Nm) Nn = \text{prf } (\text{+-N } Nm Nn)$
where postulate $\text{prf} : N\ (m + n) \rightarrow N\ (\text{succ}_1\ m + n)$
 $\{-\# \text{ ATP prove prf \#-}\}$

Combined Co-Inductive Proofs in FOTC

Example (The map-iterate property)

The `map-iterate` property is a common example to illustrate the use of co-induction.

Combined Co-Inductive Proofs in FOTC

Example (The map-iterate property)

The map-iterate property is a common example to illustrate the use of co-induction.

First-order versions of the map and iterate functions.

postulate

```
map      : D → D → D
map-[]   : ∀ f → map f [] ≡ []
map-::   : ∀ f x xs → map f (x :: xs) ≡ f · x :: map f xs.
{-# ATP axiom map-[] map-:: #-}
```

postulate

```
iterate   : D → D → D
iterate-eq : ∀ f x → iterate f x ≡ x :: iterate f (f · x)
{-# ATP axiom iterate-eq #-}
```


Combined Co-Inductive Proofs in FOTC

Example (The map-iterate property)

The bisimilarity relation (equality between potentially infinite terms).

postulate

$_ \approx _ : D \rightarrow D \rightarrow \mathbf{Set}$

$\approx$ -out:

$\forall \{xs\ ys\} \rightarrow xs \approx ys \rightarrow$

$\exists [x'] \exists [xs'] \exists [ys']$

$xs \equiv x' :: xs' \wedge ys \equiv x' :: ys' \wedge xs' \approx ys'$

$\approx$ -coind :

$(B : D \rightarrow D \rightarrow \mathbf{Set}) \rightarrow$

$(\forall \{xs\ ys\} \rightarrow B\ xs\ ys \rightarrow$

$\exists [x'] \exists [xs'] \exists [ys']$

$xs \equiv x' :: xs' \wedge ys \equiv x' :: ys' \wedge B\ xs'\ ys') \rightarrow$

$\forall \{xs\ ys\} \rightarrow B\ xs\ ys \rightarrow xs \approx ys$

{-# ATP axiom $\approx$ -out #-}

Combined Co-Inductive Proofs in FOTC

Example (The map-iterate property)

The map-iterate property asserts that the potentially infinite lists `map f (iterate f x)` and `iterate f (f · x)` are equals.

To prove the map-iterate property, we use the $\approx$ -coind rule on a particular bisimulation B (Giménez and Casterán 2007), and the hypotheses required by $\approx$ -coind are automatically proved by the ATPs.

Combined Co-Inductive Proofs in FOTC

Example (The map-iterate property)

$\approx\text{-map-iterate} : \forall f\ x \rightarrow \text{map } f (\text{iterate } f\ x) \approx \text{iterate } f (f \cdot x)$

$\approx\text{-map-iterate } f\ x = \approx\text{-coind } B\ h_1\ h_2$

where

$B : D \rightarrow D \rightarrow \mathbf{Set}$

$B\ xs\ ys =$

$\exists[y]\ xs \equiv \text{map } f (\text{iterate } f\ y) \wedge ys \equiv \text{iterate } f (f \cdot y)$

`{-# ATP definition B #-}`

postulate

$h_1 : \forall \{xs\ ys\} \rightarrow B\ xs\ ys \rightarrow \exists[x']\ \exists[xs']\ \exists[ys']$

$xs \equiv x' :: xs' \wedge ys \equiv x' :: ys' \wedge B\ xs'\ ys'$

`{-# ATP prove h1 #-}`

postulate $h_2 : B (\text{map } f (\text{iterate } f\ x)) (\text{iterate } f (f \cdot x))$

`{-# ATP prove h2 #-}`.

Apia Implementation

- Using Agda as a Haskell library

Working with a not stable API (Agda is a research system)

Apia Implementation

- Using Agda as a Haskell library

Working with a not stable API (Agda is a research system)

- Agda η -contraction

Agda performs η -contraction in the internal representation of their types. For example, the Agda internal representation of the following types are the same

$$t : \forall d \rightarrow \exists [e] d \equiv e$$
$$t' : \forall d \rightarrow \exists (_ \equiv _) d).$$

Since there is no notion of η -contraction in first-order theories, the Apia program performs an η -expansion on the Agda internal types.

Apia Implementation

- Erasing proof terms

Since there is no notion of proof term in FOL, it is necessary to erase the proof terms when translating the Agda types into TPTP.

In the translation of

$$\text{nsucc} : \forall \{n\} \rightarrow (Nn : N \ n) \rightarrow N \ (\text{succ}_1 \ n)$$

the Agda programs erase the proof term Nn .

Apia Implementation

- Erasing proof terms

Since there is no notion of proof term in **FOL**, it is necessary to erase the proof terms when translating the **Agda** types into **TPTP**.

In the translation of

$$\text{nsucc} : \forall \{n\} \rightarrow (Nn : N\ n) \rightarrow N\ (\text{succ}_1\ n)$$

the **Apia** programs erase the proof term Nn .

- Parallel ATPs invocation

From our experiments, we can conclude that the ATPs we use are complementary that is, where one ATP succeed, other ATPs fail, and the other way around.

The Automatic Theorem Provers

The overall performance of the ATPs in our formalisation of first-order theories is quite satisfactory.

ATP (total theorems: 855)	Proven thms	Unproven thms	% Success
E 1.8-001 Gopaldhara	828	27	97%
Vampire 0.6 (revision 903)	828	27	97%
Equinox 5.0 alpha (2010-06-29)	775	80	91%
SPASS 3.7	755	100	88%
Metis 2.3 (release 2012-09-27)	588	267	69%

Verification of Lazy Functional Programs

We illustrate our approach with some examples where we verify some general (co-)recursive programs and properties.

- Non-structural recursive functions
- Nested recursive functions
- Higher-order recursive functions
- Functions without a termination proof
- Unguarded co-recursive functions (e.g. verification of the alternating bit protocol)

Remark: None of the above examples can be directly formalised in **Agda** or **Coq** (they do not pass the termination checker).

Mirror: A Higher-Order Recursive Function

We prove that the `mirror` function for general trees (tree structures with an arbitrary branching) is an involution.

Mirror: A Higher-Order Recursive Function

We prove that the `mirror` function for general trees (tree structures with an arbitrary branching) is an involution.

We extend the FOTC-terms with a constructor for trees

```
postulate node : D → D → D
```

We mutually define predicates for total and finite trees and forests

```
data Forest where
```

```
  fnil  : Forest []
```

```
  fcons : ∀ {t ts} → Tree t → Forest ts → Forest (t :: ts)
```

```
data Tree where
```

```
  tree : ∀ d {ts} → Forest ts → Tree (node d ts)
```

ATP axioms

```
{-# ATP axiom fnil fcons tree #-}
```

Mirror: A Higher-Order Recursive Function

The mirror function

postulate

```
mirror      : D
mirror-eq   :  $\forall d\ ts \rightarrow$ 
              mirror  $\cdot$  node d ts  $\equiv$ 
              node d (reverse (map mirror ts))
```

ATP axiom

```
{-# ATP axiom mirror-eq #-}
```

The property

```
mirror-involutive :  $\forall \{t\} \rightarrow \text{Tree } t \rightarrow \text{mirror} \cdot (\text{mirror} \cdot t) \equiv t$ 
```

The proof is by pattern matching on the mutually defined totality predicates for trees and forests.

Mirror: A Higher-Order Recursive Function

The proof

Base case:

```
mirror-involutive (tree d fnil) = prf
  where postulate prf : mirror · (mirror · node d []) ≡ node d []
          {-# ATP prove prf #-}
```

Inductive case:

```
mirror-involutive (tree d (fcons {t} {ts} Tt Fts)) = prf
  where
    postulate
      prf : mirror · (mirror · node d (t :: ts)) ≡ node d (t :: ts)
      {-# ATP prove prf helper #-}
```

The local hypothesis helper follows by induction on forests:

```
helper : ∀ {ts} → Forest ts →
  reverse (map mirror (reverse (map mirror ts))) ≡ ts
```

Conclusions

- We defined **FOTC**, a **first-order** programming logic suitable for reasoning about mainstream **lazy** functional programs including those that use **general** recursion
- We chose a mature system as our interactive proof assistant to formalise our programming logic. We use **Agda's proof engine** for writing our proofs and we use it as **logical framework**.
- To deal with low level reasoning (equational reasoning and first-order reasoning), we used off-the-shelf ATPs
 - We extended **Agda** with the ATP-pragma
 - We wrote the **Apia** program which translated our **Agda** representation of first-order formulae into the **TPTP** and it calls the ATPs to try to prove the translated conjectures

Future Work

- Proof term reconstruction

We would like to modify our **Apia** program so that it can return witnesses for the automatically generated proofs so that they can be checked by **Agda**.

- Polymorphism

We need to support polymorphism if we want to deal with a larger fragment of **Haskell**-like languages.

- Connection to Satisfiability Modulo Theories (SMT) solvers

An interesting improvement to our **Apia** program would be to integrate SMT solvers into it.

Thanks!

References

- Abel, Andreas, Coquand, Thierry and Norell, Ulf (2005). Connecting a Logical Framework to a First-Order Logic Prover. In: *Frontiers of Combining Systems (FroCoS 2005)*. Ed. by Gramlich, B. Vol. 3717. *Lecture Notes in Artificial Intelligence*. Springer, pp. 285–301.
- Bove, Ana, Krauss, Alexander and Sozeau, Mattieu (2012). Partiality and Recursion in Interactive Theorem Provers. An Overview. Accepted for publication at *Mathematical Structures in Computer Science*, special issue on DTP 2010.
- Dybjer, Peter (1985). Program Verification in a Logical Theory of Constructions. In: *Functional Programming Languages and Computer Architecture*. Ed. by Jouannaud, Jean-Pierre. Vol. 201. *Lecture Notes in Computer Science*. Springer, pp. 334–349.
- (1990). Comparing Integrated and External Logics of Functional Programs. *Science of Computer Programming* 14, pp. 59–79.

References

- Dybjer, Peter and Sander, Herbert P. (1989). A Functional Programming Approach to the Specification and Verification of Concurrent Systems. *Formal Aspects of Computing* 1, pp. 303–319.
- Giménez, Eduardo and Casterán, Pierre (2007). A Tutorial on [Co-]Inductive Types in Coq. URL: <http://coq.inria.fr/documentation> (visited on 29/07/2014).
- Sutcliffe, Geoff (2009). The TPTP Problem Library and Associated Infrastructure. The FOT and CNF Parts, v3.5.0. *Journal of Automated Reasoning* 43.4, pp. 337–362.